Enforcing The Chinese Wall Security Policy in Security-Enhanced Linux

by

Calum Cardownie

202248259

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project

Department of
Computer and Information Sciences

March 2026

Abstract

The Chinese Wall Policy is a unique security policy in the sense that it involves enforcing dynamic restrictions, where history of the subject's operations dictate their privileges for the rest of their runtime. This paper documents the process that led to the first documented successful implementation of this policy model in Security-Enhanced Linux. The complex security system introduced by the operating system was leveraged to push the implementation to its limits. Agile-like stages ensured consistent progress, and a strong final phase completed a robust Chinese Wall Daemon, complete with policy modules to enforce the dynamic Chinese Wall. The Wall's strengths and weaknesses were evaluated in assessing the final product, and its introduction to Security-Enhanced Linux aims to establish the groundwork for more unique policies.

Acknowledgements

I would like to thank my supervisor, Ross Horne, and my second marker, Marc Roper, whose combined assistance helped shape the outcome of the project.

Thank you to my friends, family, and partner, who supported my work and encouraged me throughout the project.

And of course, credit to the Livingstone Tower.

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Project Aim . . . 1
1.3 Research Question & Objectives . . . 2
1.4 Report Format . . . 2

2 Literature Review **3**
2.1 The Chinese Wall Policy . . . 3
2.2 Chinese Walls in Adjacent Systems . . . 4
2.3 Security-Enhanced Linux . . . 7
2.4 Origin and Role of Security-Enhanced Linux . . . 10

3 Specification **11**
3.1 Methodology . . . 11
3.2 Project Stages . . . 11
3.3 Analysis . . . 12
3.4 Contingency Plan . . . 13
3.5 Requirements . . . 13

4 Design **15**
4.1 Policy Design . . . 15
4.2 System Design . . . 16
4.3 Assisting Programs . . . 18

5 Implementation **19**
5.1 Virtual SELinux Environment . . . 19
5.2 Technology Selection . . . 19
5.3 Version Control . . . 20
5.4 Policy Modules . . . 21
5.5 Internal Packages . . . 22

6 Verification & Validation **26**
6.1 Testing & Verification . . . 26
6.2 Validation . . . 27

7 Results & Evaluation **28**
7.1 Evaluation Process . . . 28
7.2 Results of Evaluation . . . 28
7.3 Returning to the Objectives . . . 29

8 Discussion & Reflection **30**
8.1 Interpreting the Results . . . 30
8.2 Reflection . . . 30
8.3 Challenges . . . 30
8.4 Limitations . . . 31
8.5 Future Work . . . 31

9 Conclusion **32**

A Appendix - Red Hat **34**
A.1 Red Hat Details 34
A.2 Virtualised Red Hat system specifications 34

B Appendix - Design **35**
B.1 Auditd logs 35
B.2 Auditd SYSCALL log 35
B.3 CWD Design Diagrams 36

C Appendix - Implementation **38**
C.1 Full Subject Policy Module (beta_rw) 38
C.2 Chinese Wall Daemon Policy Module 39

D Appendix - Testing Suite **42**
D.1 Integration Testing Daemon 42
D.2 Integration Testing Plan 43

E Appendix - Test Results **44**
E.1 Before Integration Testing 44
E.2 After Integration Testing 44
E.3 Enforcement Failure Graph 44
E.4 Unit Testing Coverage 45

1 Introduction

1.1 Background

"The more you know, the less you can share" serves as the basis for the Chinese Wall Model. The policy model deems that a subject (daemon/process) in the system shall lose its read/write privileges as it gathers more information [1]. Conflict of Interest (`CoI`) classes are used to dictate restrictions, with the goal of the policy to stop the subject from sharing sensitive information between objects (files) that are conflicting with each-other. The dynamic nature of the Chinese Wall makes it a specifically intriguing security policy, in stark contrast from typical static security policies.

Security-Enhanced Linux (SELinux) is a well-known solution to all sorts of security concerns threatening servers and personal devices. SELinux implementing *Red Hat* have solidified themselves as the industry choice for secure hosting solutions, being a leading operating system used by over 90% of Fortune 500 companies for its quality privacy guarantee [2]. The flexible set of security controls brought by SELinux brings great freedom to development of specialised policies.

1.2 Project Aim

The project aims to prove the possibility of the Chinese Wall (Brewer-Nash) Model in SELinux, leveraging its *Mandatory Access Control* (MAC) system. The system introduces specialised labels for each subject and object type, that allow precise control over their access to system operations.

The Chinese Wall has received a distinct lack of attention in the space surrounding SELinux, considering its notoriety in surrounding areas. While providing an implementation for SELinux, this report delves into the use cases of this specific policy, where it has succeeded in the past, and what its inclusion in SELinux facilitates.

Previous research into both Chinese Wall use cases and SELinux was factored in the design process, along with hands on experimentation and prototyping of different solutions. The development of the Chinese Wall in SELinux aims to open possibilities of dynamic policies in the otherwise static SELinux, and potentially lead the way in future security solutions.

To enforce this policy model in SELinux, Policy Modules (PMs) must be compiled into the kernel via the security server and work in tandem with the Chinese Wall Daemon, developed to be capable of enforcing dynamic label alterations to monitored subjects. A lattice based system is used to transition

subject labels based on their read and write history accordingly, which should suffice to handle all cases of the Chinese Wall. Finally, this allows the SELinux Access Vector Cache (AVC) to handle the restrictions on a kernel level, successfully locking the system down to `CoI` breaches once intervened.

1.3 Research Question & Objectives

Research Question:

Can SELinux's Mandatory Access Control system facilitate the Chinese Wall Policy?

Objectives to Achieve:

1 Develop a set of Policy Modules, capable of enforcing a static Chinese Wall on subjects confined by SELinux.

2 Implement a method of dynamically altering labels, allowing SELinux to enforce the Chinese Wall policy.

3 Conclude why development in this area may be lacklustre, by analysing the difficulties and limitations that arose with development.

1.4 Report Format

The report is structured in a sequence appropriate to the process that was undertook addressing this challenge. Chapter 2 introduces background knowledge and similar work to set the basis for the project. Chapter 3 summarises the methodology and order of approach to the project. Chapter 4 details design intricacies, and Chapter 5 goes into depth of implementation to these designs. Chapter 6 will demonstrate testing and validation, and Chapter 7 will cover the results. Chapter 8 will include a discussion and reflection, and finally Chapter 9 will conclude the paper.

2 Literature Review

2.1 The Chinese Wall Policy

2.1.1 The Brewer-Nash Model

The Chinese Wall - or Brewer-Nash - Model is a security model that aims to prevent information flow between datasets that are in a `CoI` relation [1]. The Brewer-Nash Model provides a mathematical formalisation of the larger Chinese Wall policy. As opposed to the classic multi-level security (MLS) Bell-LaPadula Model, which enforces *"no read up, no write down"* [3], the Chinese Wall Policy allows for information to flow freely until a subject attempts to read from a `CoI` object. Upon this access, the system shall dynamically revoke their write permissions to other objects in the `CoI` group, creating a "wall" in-between them. Objects are considered in a hierarchical manner, (see Figure 1). Consider the following scenario:

A data server stores information on Bank A and B, and Tech Company A and B. These companies are in a `CoI` relation with their respective counterparts. If a subject approaches the dataset, they are initially able to access any object. As far as the system is concerned, the subject is free, as it has no prior knowledge of any company. Suppose the subject has read from Bank A, the situation develops - the subject now possesses information that, deemed by the Chinese Wall, should not be allowed to reach Bank B, as they in a `CoI` relation with Bank A. The Wall revokes the write access between the subject and Bank B. Now information *cannot* flow from Bank A to B.

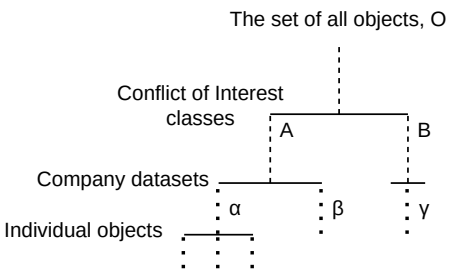


Figure 1: The composition of objects

The system is constructed to definitively prevent this leakage scenario. It should also be noted that once information from Bank A is read, it cannot be shared to Tech Company A or B, even though they don't share a `CoI` relation with Bank A. In that scenario, a different subject could read Bank A's information from Tech Company A and write it to Bank B, as they are not in conflict, exploiting the rules of the Chinese Wall. This means that the Brewer-Nash Model is a considerably restrictive policy, as once you read from an object you are essentially tied to that dataset.

2.1.2 Intricacies of the Chinese Wall

Brewer & Nash's rules demonstrated the Chinese Wall as a mathematical concept, rather than a policy. They broke it down into the following rules, influenced by Bell-LaPadula:

- **Simple Security Property**: Subject `S` can read object `O` only if `O` is in the same Company Dataset (`CD`) as an object already read by `S`, or if `O's CD` is in a different `CoI` group from all `CD`s of objects already read by `S`.
- ***-Security Property (Star Property)**: Subject `S` can write to an object `O` only if `S` can read `O` **and** all objects `O'` read by `S` are in the same `CD` as `O`.

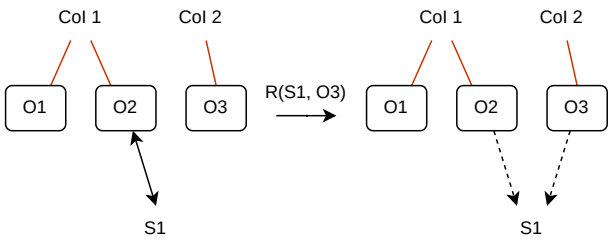


Figure 2: Demonstration of the *-property

Observe Figure 2, where a full line represents read/write access and a dotted line represents read access; once a read is performed on O3, the **-property* dictates that subject S1 must lose its write access to O2.

The Brewer-Nash Model suffers from a lack of clarity in some areas, leaving readers split on interpretations of certain definitions in the paper. The *-property for one, leaves it unclear if write access implies read access. As such, this report will follow the interpretation that write access does imply read access, with the exception of sanitised objects, which every subject can read as they are specifically deemed "clean".

After the Brewer-Nash Model was communicated, another paper was published once again by Brewer & Nash, but also T.Y. Lin, who presented them with potential improvements on their model after their original paper, leading to the *Aggressive Chinese Wall Model* [4]. This idea, trying to disprove the assumption that `CoI` is an equivalent relation, has since been accepted into the mainstream. This paper will not follow Lin's model, instead focusing on the traditional Brewer-Nash.

2.2 Chinese Walls in Adjacent Systems

To understand the effect that a new Chinese Wall can have, and how measures could be taken to implement it, other ventures in adjacent systems should be investigated and evaluated. Unix, Cloud, Workflow Management and Finance will be discussed in the following.

2.2.1 Foley's Unix Model

A Chinese Wall Model implementation was attempted in standard Unix, leveraging its *Discretionary Access Control* (DAC) security system also adopted by Linux and macOS. The 1997 Unix proof by S.N. Foley built upon T.Y. Lin's Aggressive Chinese Wall Model, illustrating the possibility of its creation in with DAC [5], featuring a `CoI` matrix that detailed a non-transitive version of `CoI` relations. Foley's concept focused on the restriction of users, preventing a user of the computer from spreading information enforced via their user ID and group ID. DAC is defined such that objects and their privileges are managed by their creator, and the super-user can access and alter anything. DAC enforces based on the user ID and any groups with which they belong. While DAC brings about great flexibility, the model's reliance on ownership often leads to uncontrolled privilege propagation. It is known to be difficult to measure the security of the DAC model, and in some cases entirely undecidable [6].

2.2.2 Cloud Computing

Cloud Computing involves many Virtual Machines (VMs) running on one core system, with the virtual machine monitor, or *hypervisor* acting as the delegating force of the computer's hardware to the VMs. Each VM is a separate operating system (OS), completely independent of the other VMs on the host. The Chinese Wall has been implemented on this level against various threat models.

2.2.2.1 sHype Hypervisor

The sHype mandatory access control system was added to the Xen hypervisor to defend against information sharing between VMs by R. Sailer *et al* [7]. The hypervisor prevents inter-VM communication by utilising kernel hooks, intercepting and evaluating every attempt to communicate across VMs. Whilst claiming to be a Chinese Wall, the sHype architecture does not truly capture the notion of Brewer & Nash's Model, instead focusing on static "Chinese Wall types". Regardless, sHype does successfully prevent against `CoI` leakages, in the case of insider activities or inter-VM attacks.

2.2.2.2 Chinese Wall Central Management System (CWCMS)

Inter-VM attacks can happen if a VM on the system is breached and can allow a targeted attack on another VM or even a compromised hypervisor. To eliminate the possibility of inter-VM attacks

from competitors, a different Chinese Wall security model was implemented. The CWCMS, operating above the Kernel-based Virtual Machine[1] level via installed control agents, with a sole focus of actively preventing `CoI` breaches was designed by Tsai *et al* [8]. Once a VM in a `CoI` group is running on a physical host, the Wall prevents other VMs belonging to the same `CoI` group from being scheduled on the same machine in advance. This is a pre-arranged decision made by graph colouring, as opposed to sHype which simply attempts to deploy it on different systems until one is satisfactory. This effectively eliminates the risk of inter-VM attacks in these systems. Similar to sHype, CWCMS also lacks dynamic label profiling, resolving to a statically enforced, yet successful, policy model.

2.2.2.3 Security-Awareness VMs Management Scheme (SVMS)

Finally, a dynamic Wall was created in Yu *et al*'s SVMS system [9]. SVMS aims to eliminate intrahost *side channel attacks*, where malicious users extract private information by reading from shared data caches on the host machine. Yu *et al* address this with two techniques: prevent simultaneous execution, and a more unique advance with VM placement and migration solutions based on VM traces. The VM tracing creates a dynamic policy model faithful to the Brewer-Nash Model, and Yu *et al* prove that the rules meet the simple-security property and *-security property.

All models, while different, agree that the *Chinese Wall* for Cloud involves preventing conflicting resources from running at the same time. Whilst not exactly cohering to the Brewer-Nash Model, they all manage to succeed in their endeavours to create a Chinese Wall.

2.2.3 Workflow Management

Beyond virtualised environments, the Chinese Wall Model has also successfully been implemented in Workflow Management Systems (WfMS), which are intrinsically network-based applications, potentially hosting multiple conflicting company datasets. As such, a secure WfMS must implement security features, specifically targeting information confidentiality as a main point of effort. The Chinese Wall Model can be leveraged to prevent `CoI` leakage in this environment. The authors proposed and implemented a *Chinese Wall Security Policy Specification Language* in Java [10]. The paper mentions that role based access control is unable to implement general cases of the model, as it has issues implementing the Brewer-Nash simple security property in dynamic contexts, supporting the premise that the nature of the Chinese Wall is difficult to implement with standard methods.

[1] KVM allows the Linux kernel to act as a hypervisor.

2.2.4 Financial Institutions

Chinese Walls serve as a pillar of compliance in the financial sector. The ethical policy, more commonly just referred as *The Wall* in finance, is a commonplace measure that should theoretically and practically prevent misuse of information leakage in banks [11] and other financial management firms. Thus, the Wall is more attuned to person-level restrictions as opposed to system-wide enforcement as is present in modern considerations of the Chinese Wall. While entirely different from modern systems, it helps to understand the origin of the Wall, and that it aimed to prevent `CoI` leakage of strictly confidential information.

2.3 Security-Enhanced Linux

Understanding the properties and behaviour of SELinux was key in approaching the project, thus, it is important to discuss the systems brought in place, and the solutions they approach.

2.3.1 Mandatory Access Control System

SELinux implements a MAC system, providing a policy-driven security layer extended beyond the classic DAC. Utilisation of labels - applied to both objects and subjects - and verbose policy modules provide the OS great freedom in restriction and enforcement. The *Security Context*, or *label* of an object/subject takes the form `user_u:role_r:type_t:s0` (see Figure 3) and can be viewed with the `-Z` flag when using `ls` or `ps`[2].

```
/home/testgrounds/objects ❯ ls -Zl | grep alpha_logs
-rw-rw-rw-. 1 cal cal unconfined_u:object_r:alpha_t:s0     28 Mar 20 00:24 alpha_logs
      Standard Unix DAC          SELinux MAC Security Context
```

Figure 3: Security Context of an object

The Security Context consists of the fields:

- **User** (`user_u`): Used to give the confine specific Linux users to a class, granting them simple privilege differences. Examples include `guest_u`, `user_u` or `staff_u`.
- **Role** (`role_r`): Used to confine users to roles, implementing role based access control.
- **Type** (`type_t`): Used to restrict subjects based on their domain type. The most specific field a policy can target.

[2] `ls` and `ps` are standard Unix commands to list objects and processes respectively.

- **Level** (`s0`): Used in MLS and Multi-Category Security (MCS) contexts to separate subjects and objects based on classification/category. Often used to isolate VMs and containers from sharing access to the same resources.

A simple policy may be that an object of type `http_t` can only be operated on by a subject of type `apache_process` and prohibits `sshd_t`. This provides strong mitigation against broken access control damages, as subjects can only access objects they are confined to by SELinux's security server. Were a subject hijacked, it is still only restricted to the domain it originally had on the system.

2.3.2 Security Architecture

Being based on the Flux Advanced Security Kernel (FLASK) security architecture for flexible non-discretionary access controls [12], the architecture of SELinux is split into 2 main bodies:

- Policy enforcement via the Access Vector Cache.
- Policy decision-making via the Policy Repository and Security Server.

The AVC serves as the main denial body, instantly able to deny a process on a cache hit. On a cache miss, the kernel briefly blocks the process as the syscall is carried out and the AVC enforces the denial/allows the syscall [13] (see Figure 4).

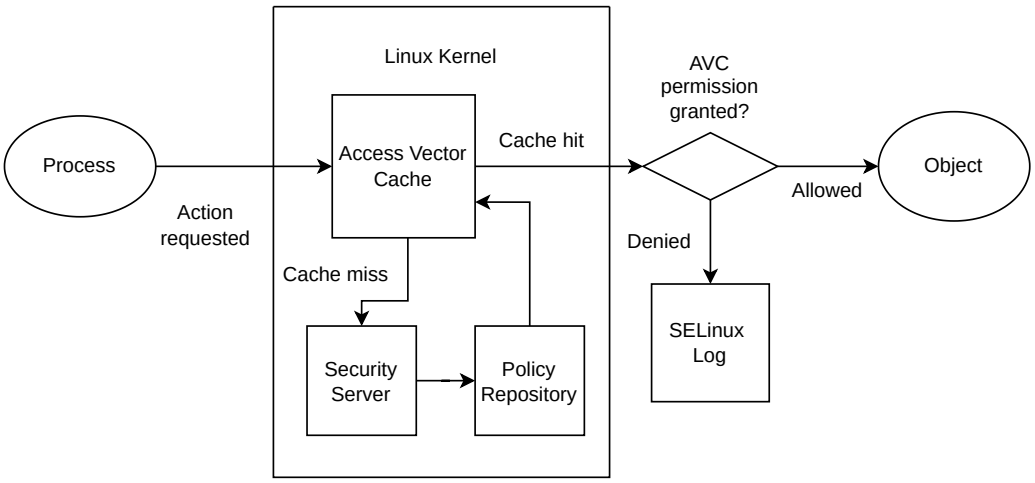


Figure 4: Primary SELinux Components

2.3.3 Policy Module System

The PMs that SELinux provides are solely controlled by the super-user. Red Hat 9 by default has 438 modules (verified on a fresh Red Hat 9.7 install), covering every domain in the system. The *policycoreutils-devel* package introduces developer tools to install and alter PMs on the system. Calling the provided `sepolicy generate --init` from the command line generates boilerplate for the policy including:

- Type Enforcement *(.te)* file: Unique syntax notates dependencies and defines rules, introduced to grant privileges to the label.
- File Contents *(.fc)* file: Details the file labelling rules for the policy.
- RPM Spec *(.spec)* file: Packages the policy to be installed into the kernel with the RPM package manager, including labelling, the interface file and associated man page of the policy.
- Interface *(.if)* file: Defines the interfaces for the types generated in the type enforcement file, allowing policies to reference each other.
- Shell Script *(.sh)* file: A script to be run by the super-user to compile and install the policy module. Under the hood, various commands are executed to complete installation.

Once installed, PMs are persistent and restrict based on their domain while SELinux is in *Enforcing* mode. SELinux also supports a *Permissive* mode, where denials are logged but not enforced. Permissive mode is integral for testing and development, but at the cost of risk in the system. The command includes a permissive line that must be removed from the *.te* file for the same reason.

Compiling modules without the template from sepolicy generate, a policy module must be installed with `checkmodule`, `make -f /usr/share/selinux/devel/Makefile` & `semodule -i`.

2.3.4 Multi-Level Security in SELinux

The Bell-LaPadula Model has previously been implemented in standard SELinux. Upon enabling and reboot, a full filesystem relabel is applied according to their MLS clearance level (top-secret, confidential...). The implementation mainly utilises the level field to restrict them based on subsections within that range of confidentiality. It is a MAC policy like the Chinese Wall, but is entirely static at runtime.

2.4 Origin and Role of Security-Enhanced Linux

2.4.1 The NSA's Creation of SELinux

SELinux is adopted by many distributions of Linux, including Red Hat Enterprise Linux (RHEL), Fedora and even mobile operating system giant, Android. The origin of SELinux can be attributed to the *National Security Agency* (NSA)'s workings on a series of books called the Rainbow books - a series of Computer Security standards and guidelines by the United States Government. They published one and created a second (unpublished) book that included a formal SELinux model and associated evaluation evidence prototype. SELinux was a demonstration that mandatory access control was viable and could be added to the Linux kernel. The NSA, along with NAI Labs were responsible for using FLASK to integrate MAC controls into the Linux kernel [14][15].

2.4.2 Inclusion of SELinux in the mainline Linux kernel

Linux, as an OS, was able to adopt this because of its commitment to software freedom and ethical use of technology, originally being an open-source derivation from the Unix that was locked down by AT&T's IP. Linus Torvalds combined his Linux kernel with the GNU system tools by the GNU Project (Richard Stallman and various developers) to create GNU/Linux [16], and was built under the *GNU General Public Licence*, dictating that all code in the system had to be free and accessible. This open-source philosophy was a key factor enabling SELinux's integration into the mainline kernel in the 2.6 series in December 2004 [17].

2.4.3 Red Hat Enterprise Linux

The impact of Linux is huge, with 85% of smartphones and 60.9% of websites worldwide powered by the open-source OS [18] [19]. Tech giant IBM produced RHEL has a 33% market share in worldwide operating systems (see Appendix A) [20], no doubt because of their commitment to reliability, stability and security.

As one of many distributions of Linux, RHEL is unique as a subscription based distribution, with a target audience of companies expecting nothing less than the finest. SELinux is clearly a lucrative avenue for development, and the introduction of new and unique security policies could yield various results.

3 Specification

3.1 Methodology

The decision to adhere to a methodology early in development helps set up consistent and satisfying progress over the life of the project. As this project was undertaken as a solo developer, the methodology was not as strictly enforced as a team setting. Still, the project lifecycle followed is closely attuned to the agile principle.

3.1.1 Agile

Agile methodology focuses on multiple iterative stages, constantly evaluating the progress and direction of the project, and tuning decisions to reach the solution. As SELinux is a system with predefined capabilities and limitations, various different methods were used throughout to probe the boundaries and prototyping was updated accordingly. Experimentation in SELinux was carried out simultaneously alongside research, allowing accumulated insight to be backed by personal experience, and a fuller and more satisfying solution.

This agile process was accompanied by a simple TUI Kanban board, easily conveying the stage of the project as well as next steps and currently occupied tasks. This allowed for a more fulfilling lifecycle of an individual idea.

3.2 Project Stages

3.2.1 Understanding SELinux & Brewer-Nash

This stage included both researching and experimentation of the main topics. A VM was set up that was used to deploy and test throughout the project. The boundaries of SELinux were to be tested, with different avenues being evaluated and documented. The complexities and nuances of Brewer-Nash were studied; proving more complex than they first seemed.

3.2.2 Static Chinese Wall

The first milestone of the project involved creating a static implementation of the Wall. This stretch aimed to prove that the SELinux type enforcement was expressive and granular enough to constrain a subject to specific allowances and denials. The phase was marked complete with the necessary

policy modules installed to restrict a set of subjects from accessing certain objects, mimicking the Chinese Wall and using simple test daemons to confirm its success.

3.2.3 Dynamic Chinese Wall

The longest stage, involved achievement of the dynamic Chinese Wall by way of creating the *Chinese Wall Daemon*, (*CWD/cwalld*) and its surrounding policy modules. This phase closely embodied the agile methodology, as the daemon was built step by step, on top of the already working static wall policies.

3.2.4 Testing & Evaluation

This phase involved testing the constraints and abilities of the final product. Suitable testing measures were all applied to effectively evaluate functionality and overall success of the project. The entire project journey was reflected upon, and limitations of the end product were assessed.

3.3 Analysis

To enforce a dynamic security policy, some process must be constantly running to be capable of enforcing this dynamism. It was determined that the most effective way to leverage SELinux's MAC system was to change the type field of the security context on active subjects in the system. The user, role and level fields of the subjects and objects are not sufficient to enforce a security policy, as flexible policy modules could not be compiled to enforce restrictions based on those fields.

Policy modules are unable to be added to the SELinux security server at runtime, as it requires a complete re-compile of the security server and kernel. This operation is capable of taking anywhere between 4 and 8 seconds - unrealistic at runtime for a background process. Thus, policy modules were identified to be impossible to install while the system is running.

Further experimentation on the properties of the type field of the label led to the philosophy that became the main premise of the CWD: pre-compiled policy modules for all scenarios, and a daemon that would alter the labels of subjects at runtime according to their history of operations. This daemon also needed to be fully integrated into SELinux's system, running under its own `cwalld_t` type.

The alteration of subjects' labels also required a brief termination of the subject, as the security

context is granted upon startup, so changing the label would require a restart at runtime. The CWD would have to do execute this procedure at will.

3.4 Contingency Plan

As this project is venturing into unknown territory, a contingency plan had to be produced to ensure a rewarding outcome was still delivered if the primary goal was unreachable. The contingency plan for the CWD was made in the case of dynamic labels being unfeasible and involved the CWD acting as an auditing daemon, auditing when breaches of the Chinese Wall had been detected, but not enforcing.

3.5 Requirements

Requirements specified early in the agile cycle follow the priority categories principle. They are split into 4 categories:

- **Must** have - features that must be implemented to achieve the goal.
- **Should** have - features that should be implemented to develop the solution.
- **Could** have - features that could be implemented but would not be prioritised.
- **Won't** have - features that were considered but decided against.

These requirements were solidified after considering the required functionality of the product in depth. While the requirements were decided from the beginning, the means of completing them was left ambiguous, in line with agile structure. Functional requirements were settled on to decide exactly what the product needed to achieve (see Table 1), whilst non-functional requirements detailed high level requirements of the product (see Table 2).

Code	Requirement	Priority
FR1	`cwalld` must alter necessary configurations in the system before running.	Must
FR2	`cwalld` must monitor all operations made by subjects across a given directory.	Must
FR3	`cwalld` must make decisions based on subject label and object label aligned with the Chinese Wall Model.	Must
FR4	`cwalld` must alter the labels of subjects at upon call.	Must
FR5	`cwalld` must be accompanied by policy modules for objects.	Must
FR6	`cwalld` must be accompanied by policy modules for subjects.	Must
FR7	`cwalld` should be confined by SELinux with its own custom policy.	Should
FR8	`cwalld` should run as a service in the system with a service declaration file.	Should
FR9	`cwalld` should write all operations to a log.	Should
FR10	`cwalld` should include a program to follow and display the log in real time.	Should
FR11	`cwalld` could automatically create necessary policy modules for its runtime.	Could
FR12	`cwalld` could automatically install necessary policy modules before running.	Could
FR13	`cwalld` could dynamically enforce the Chinese Wall Model given a set of `CoI` classes as input.	Could
FR14	`cwalld` won't enforce the Chinese Wall Model system-wide.	Won't

Table 1: Functional Requirements

Code	Requirement	Priority
NFR1	`cwalld` shall solely be controlled by the super-user.	Must
NFR2	`cwalld` shall run as a daemon in user-space of the computer.	Should
NFR3	`cwalld` shall be written with structural, intentional and idiomatic code.	Should
NFR4	`cwalld` shall be a good Unix citizen, following classic system conventions.	Should
NFR5	`cwalld` shall function as intended across both x86 and ARM architecture.	Should
NFR6	`cwalld` shall act atomically, ensuring the Chinese Wall is instantaneous.	Won't

Table 2: Non-Functional Requirements

4 Design

4.1 Policy Design

To demonstrate the Chinese Wall Model in SELinux, the minimum number of labels and objects necessary to capture all relevant cases were simulated. The simulation composed 3 object classes: two - alpha (α) and beta (β) - in a `CoI` relation, and one that was not, gamma (γ).

4.1.1 Object Policy Modules

A unique PM had to be installed for each different `CoI` type. The object PMs were simple, as they only needed to introduce the type to the system. This allowed the subject PMs to reference them as necessary. Sanitised is introduced as an object type that all subjects can read from, containing information deemed "safe" by the system. The following types were to be introduced:

- `alpha_t`
- `beta_t`
- `gamma_t`
- `sanitised_t`

4.1.2 Subject Policy Modules

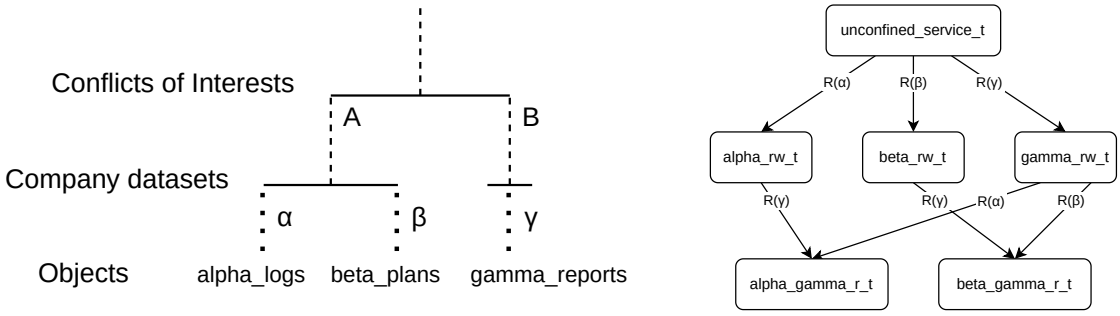
By default, SELinux restricts all actions that aren't granted in some PM. Thus, design of subject PMs must explicitly grant the subject all operations they require. Six labels are designed in Table 3 below. It should be noted that `unconfined_service_t` is a pre-existing label in SELinux for services that haven't been previously labelled.

Label	`alpha_t`	`beta_t`	`gamma_t`	`sanitised_t`
`unconfined_service_t`	r/w	r/w	r/w	r/w
`alpha_rw_t`	r/w		r	r
`beta_rw_t`		r/w	r	r
`gamma_rw_t`	r	r	r/w	r
`alpha_gamma_r_t`	r		r	r
`beta_gamma_r_t`		r	r	r

Table 3: Subject labels and their privileges

These labels may be unconventional, for instance, why is it that `alpha_rw_t` can read from `gamma_t`? SELinux is preventative; the standard Wall method of a subject *requesting* permission to access an object or subject is not possible. As such, the enforcement involves allowing the subject to access anything that it *could*, instantly altering its label afterwards. While a subject with label `alpha_rw_t`

can read from `gamma_t`, its label will instantly transition to `alpha_gamma_r_t`, revoking write access to `alpha_t` forever. Figure 5 illustrates the planned object composition and subject label lattice. 5a details the objects that compose the experiment, and their `CoI` status. 5b illustrates the transition that a subject may traverse while operating in the system.



(a) Object structure (b) Subject label transition lattice

Figure 5: SELinux Chinese Wall

4.1.3 Chinese Wall Daemon Policy Module

The CWD PM would be significantly different to the other PMs, as it would need a huge array privileges in the system, such as reading metadata, communicating with the D-Bus, and relabelling other subjects - effectively meaning it could control the MAC system. It would be created following the principle of least privilege. The standard security principle states that a subject should only have access to the required functionality that is necessary to complete their tasks. The CWD would have a `cwalld_t` label that should suffice to do just that, allow it to operate and nothing more. Keeping the CWD secure would be an important task, if system security was paramount.

4.2 System Design

4.2.1 Tailing the Audit Daemon

The central idea of the CWD surrounds tailing the log produced by *auditd*. A system daemon enabled by default in SELinux distributions and included in regular distributions, the *Linux Auditing Daemon* is capable of documenting every system call in the system, ranging from reads, writes, SELinux denials and everything else. A rule can be added to `/etc/audit/rules.d/audit.rules`, one which can make auditd log every operation in a given directory. This feature would be key in surveilling a certain directory as was outlined in FR2. The information in the auditd log includes the following information, necessary for the CWD (see Appendix B).

SYSCALL audit:

- Subject process ID
- Subject name
- Subject label
- Subject entrypoint
- Operation
- Operation success

PATH audit:

- Object path
- Object label

AVC audit:

- Specific operation denied

By parsing and combining this information at runtime, subjects in the system can be profiled, and have their operations and history precisely tracked. The logic for restrictions could then be passed to the Chinese Wall enforcement module for a decision to be made and executed.

4.2.2 Subject Label Alteration at Runtime

While a subject label can be detected at runtime with the `ps -efZ` command, this label is immutable. In SELinux, to change the security context of a subject, the label of the entrypoint[3] must be altered, allowing for the subject to transition into the new label domain once it is restarted. The entrypoint gets the *exec* version of the corresponding label, which mutates to the standard version when assigned to the subject. This means the CWD will have to maintain a connection to the system *D-Bus*, to restart daemons at will. Most system daemons are designed with graceful shutdown processes, such that a restart does not constitute a significant loss. Upon restart, a SIGTERM signal is sent to the process following up with a SIGKILL if not promptly handled.

4.2.3 Flowchart

The flowchart (see Appendix B.3.1) represents the central logic loop of the CWD, surrounding the tailing of `audit.log`. The design of this chart would lead production throughout multiple iterations.

4.2.4 UML Diagram

A UML diagram was produced, detailing the project internal structure and relationships between modules, and was repeatedly addressed and iterated upon throughout development (see Figure 6, full size in Appendix B.3.2).

[3]The entrypoint is the binary in the system that executes the subject startup sequence.

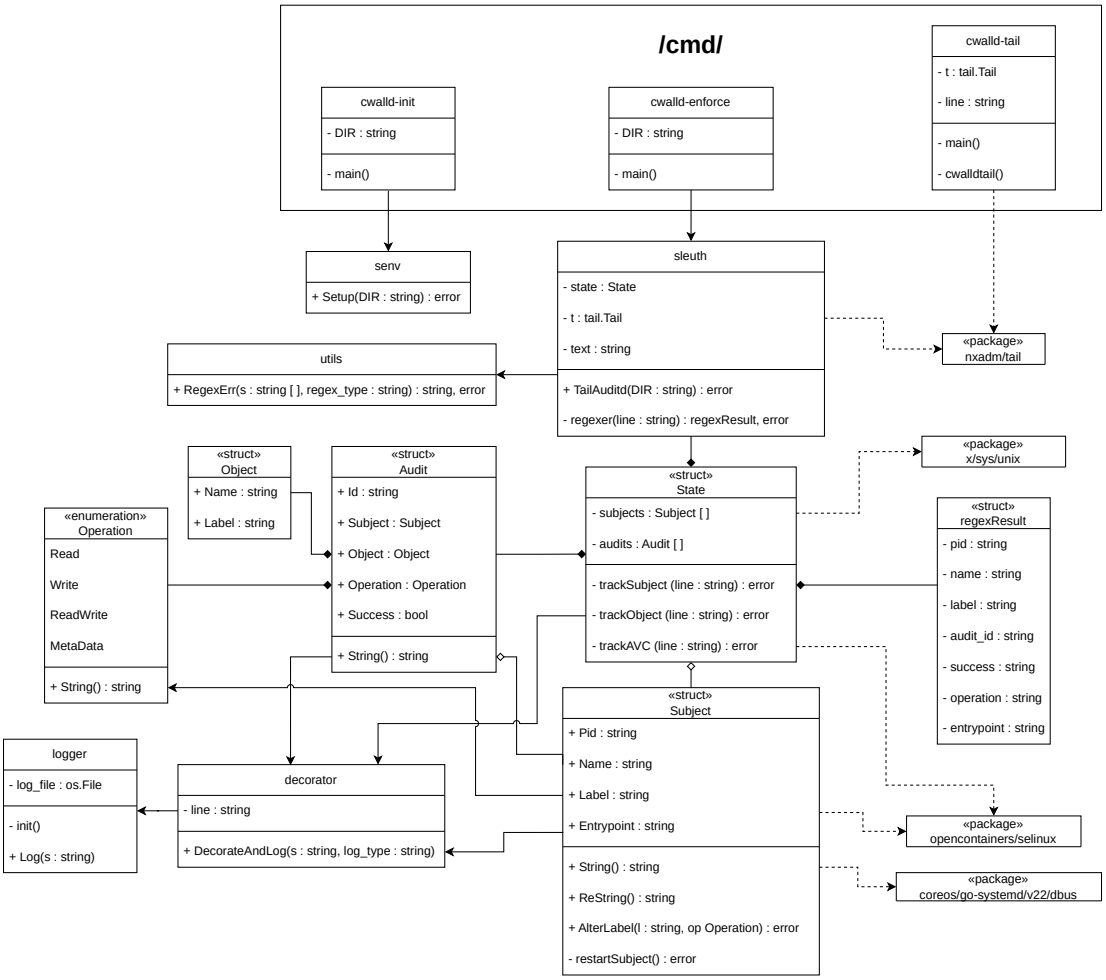


Figure 6: CWD UML diagram

4.3 Assisting Programs

As the CWD will be fully integrated into the system, assisting programs exist to fulfil FR1 and FR10.

4.3.1 cwalld-init

The CWD *Init* program should set up the audit rule that is required for the CWD to operate. It should also report any issues with the setup process.

4.3.2 cwalld-tail

The CWD *Tail* program should tail and return every line that is logged in `/var/log/cwall/cwall.log`. This allows us to track the cwalld-*enforce* process in real-time, as it tracks operations and alters labels.

5 Implementation

5.1 Virtual SELinux Environment

To develop and test on SELinux, an appropriate SELinux environment is necessary. RHEL was chosen, as it is the most commonly used SELinux distribution. Fedora was also considered; a popular fork of Red Hat skewed towards personal workstations rather than enterprise servers. As the server aspect is more likely to reap results, Red Hat was chosen. As RHEL is an enterprise product, the Red Hat Developer program was joined for free access. Once a key was acquired, a VM was created using KVM, *Quick Emulator* (QEMU) and *Virt Manager* to interface with the VM.

An *ssh reverse tunnel* allowed the virtual environment to be navigated from the command line of the host machine. RHEL 9 and RHEL 10 were both used for experimentation, RHEL 9 being used after the instance of RHEL 10 was rendered unusable during testing. System specifications can be seen in Appendix A.2.

5.2 Technology Selection

5.2.1 Language

Go *(or Golang)* was chosen for the development of the CWD, due to its strong affinity with system-level programming. Go's advantages in systems engineering range from:

- **Daemon Focus:** Go is very well-suited to daemon programming, remaining unpunishing while giving the developer great versatility and library access.
- **Performance:** As a compiled language, the flexibility to be built into a binary means minimal computational overhead at runtime, promising low-latency, high-performance applications. Furthermore, the freedom to use pointers also allows more precision in containing overhead.
- **Modernity:** Go is a modern alternative to the systems engineering environment. The focus on clean, idiomatic code with statically enforced typing discourages unnecessary complexity and promotes hands-on learning.
- **Familiarity:** Repeated experience working with the language backed the choice and supported steady progress in the venture.

5.2.2 Packages

nxadm/tail

A fork of the deprecated tail package by *hpcloud*, *nxadm* provides a Tail object that can be used to tail a specified file. Tailing a file involves reading every update that occurs, and returning them in real time, allowing a program to closely monitor a file for its own needs. The package leverages *fsnotify*[4] to instantly recognise file updates, also providing truncation and move support, allowing it to follow rotating log files with minimal configuration.

opencontainers/selinux

The *selinux* package provides Go bindings for SELinux label operations, enabling programmatic reading and manipulation of labels. It also introduces native error handling to Go, streamlining usage and debugging.

x/sys/unix

The official Go Project *sys/unix* package introduces a layer of abstraction over both x86 and ARM architecture system call codes, providing a unified interface for detecting system operations.

coreos/go-systemd

go-systemd introduces tools to communicate directly to the system D-Bus API. This allows programmatic control of operations typically managed with `systemctl`, such as starting, stopping and monitoring services on a system.

5.3 Version Control

Git was heavily used throughout the project as the primary version control measure. It allows incremental development across multiple devices, which would be critical for splitting development between RHEL and the host machine.

Git also kept a backlog of all modifications, allowing changes to be reverted if found to be problematic, especially in the brittle SELinux developer environment. This gave the project a guarantee that experimentation could flourish without risk of unrepairable damage.

[4] fsnotify is a package that can read filesystem notification APIs, using *inotify* on Linux.

5.4 Policy Modules

5.4.1 Object Policy Modules Implementation

The SELinux type enforcement system will be briefly covered, as it has an esoteric syntax.

Object PMs simply instantiate the domain for the system (see Code Block 1). The `files_types` macro is shorthand for defining the label globally as a file type.

```
1 module beta 1.0;
2
3 type beta_t;
4
5 files_type(beta_t)
```

Code Block 1: Object *.te*

5.4.2 Subject Policy Modules Implementation

PMs had to be created and compiled for $5/6$ of the labels from Figure 5b (full PM in Appendix C.1). The 6th is the predefined `unconfined_service_t`

Subject PMs include the `policy_module` macro, to implement abstracted basic requirements. They must also require every type that they reference. Notice `user_home_t` is needed to grant the subject navigational privileges across those directories.

```
11 require {
12   type beta_t; type gamma_t; type sanitised_t;
13   type user_home_t; type user_home_dir_t; }
```

The following lines are required for all subject PMs, detailing basic daemon behaviour:

```
19 init_daemon_domain(beta_rw_t, beta_rw_exec_t)
26 allow beta_rw_t self:fifo_file rw_fifo_file_perms;
27 allow beta_rw_t self:unix_stream_socket create_stream_socket_perms;
34 allow beta_rw_t user_home_t:dir { search read open write add_name };
35 allow beta_rw_t user_home_dir_t:dir { search read open write
      add_name };
```

`init_daemon_domain` is a macro for standard daemons, just shorthand for defining necessary types. Lines 26 and 27 are for communicating with *FIFO* files, D-Bus and the kernel, all necessary for standard operations. Line 34 and 35 allows the subjects to navigate directories with the `user_home_t` and `user_home_dir_t` labels. Permission to `add_name` is required as well, as writing files in SELinux periodically requires that permission.

The inclusion of these lines stress that SELinux requires *every* interaction to be granted in the system.

The second lattice layer consisted of the labels:

- alpha_rw_t
- beta_rw_t
- gamma_rw_t

```
allow beta_rw_t beta_t:file { read write getattr open };
allow beta_rw_t gamma_t:file { read getattr open };
allow beta_rw_t sanitised_t:file { read getattr open };
```

In SELinux PMs, getattr and open must be included to grant read privileges.

The third lattice layer was implemented similarly, towards the specific permissions of the two labels:

- alpha_gamma_r_t
- beta_gamma_r_t

5.4.3 Chinese Wall Daemon PM Implementation

A PM had to be constructed to encompass the runtime binary of the CWD. Once all of the functionality of the daemon was complete, SELinux was set to *permissive* mode, where all denials would be logged but not enforced. The CWD was started and all required privileges were stored, then the audit2allow command was leveraged to create a PM granting those requirements. This PM was not complete, as the final product involved behaviour that would only be executed if a denial was enforced. Such additions to the PM were found by experimentation, trial and error (see Appendix C.2).

5.5 Internal Packages

Because the CWD is a low level construction, implementation will naturally include more nuances and development details, as opposed to larger scale system integration.

5.5.1 SEnv (Security-Enhanced Environment)

Before the CWD can run, the audit rule must be initialised. The SEnv module, called by the cwalld-init program writes the rule: "-D\n-w "/home/testgrounds/" -p rwa -k cwalld" to /etc/audit/rules.d/audit.rules. The rule tells auditd to delete previous rules, then watch /home/testgrounds/, and label all reads, writes and attribute changes with the cwalld key in the audit logs. An augenrules --load is then called to apply the rules to the currently running auditd.

5.5.2 Sleuth

The core of the CWD, Sleuth maintains a reference to a single *state* struct, which holds a slice[5] of subjects, and a slice of audits.

Reading lines from the audit log is the central operation that powers the rest of the CWD. To read the audit lines, the tail must be instantiated. Flags must be set to keep reading lines, and follow/reopen new log rotations. It is also called to skip all previous text in the log before the tail was created. This should stop the CWD from attempting to enforce operations from before its startup.

Once the tail is initialised, reading from it triggers an infinite loop, and returns every new line appended. These lines can then be classified and passed into their respective function.

Before further computation, a brief check is done to see if *setroubleshootd*, or cwalld-enforce itself is involved, the former being an SELinux troubleshooting daemon that conducts checks after denials occur. Given that it is a necessary system process that operates in the enforced directory, it must be exempt from any interference. The cwalld-enforce binary itself must also be captured early, as if the CWD begins to log itself, a new audit log is created, triggering an infinite loop of CWD self computation. Both of these checks call `continue` if found, signalling to skip the rest of the loop and listen for the next log.

As the state struct holds a slice of subjects and audits, using methods means the contents can be mutated directly in memory for minimal overhead. The state struct defines the three main methods that parse logs from auditd.

```
76 func (state *State) trackSubject(line string) error { ... }
147 func (state *State) trackObject(line string) error { ... }
190 func (state *State) trackAVC(line string) error { ... }
```

These methods are always called in order due to the consistent order of logging by auditd. Each of these methods use *regular expressions* (regex) to efficiently parse the correct details from the audits. Under the hood, Go's RE2 regex engine builds *Finite Automata* to ensure $O(n)$ computation time for the capture groups, as opposed to the $O(2^n)$ complexity for the backtracking approach adopted by most other languages. The subject details can be constructed from the various submatches, and then appended to the state. Certain checks have to be made to verify if the subject is simply involved

[5] A slice in Go is a header to an underlying array.

in another audit, of which new creation is not required, and another check to see if the same subject has returned with a new PID and label.

`trackSubject()` specifically has 7 regex operations, so logic was abstracted to a seperate `regexer()` function that returns a `regexResult`. `trackObject()` and `trackAVC()` have 3 each, not enough to constitute a separate function.

There is however, another exemption to be wary of. Atomic processes, like *cat*, must be skipped entirely. As they are not running subjects, they cannot accumulate a history and be enforced upon. The way to check for atomic processes involves the `selinux.PidLabel()` function, called in `regexer()`. The role is to acquire the label of the logged subject, and whilst the label information is also in the log itself, this function can be used to skip the regex containing its label and check if the process is valid in one swoop. If the function returns an error containing "no such process", the program propagates its own error to indicate that the process is atomic and to be ignored.

Throughout all programming in the CWD, errors are propagated all the way to the `/cmd/` level, allowing the client code freedom to act accordingly upon errors. This allowed the atomic process exception to be returned to `trackSubject()`, such that it could alter its procedure as necessary. If the subject did not instantiate the audit, the PATH log would compute but not finish the audit, successfully skipping over the atomic process.

`trackSubject()` finishes by instantiating an audit. The audit also includes information about operation and success. To parse the operation, a procedure must be followed to convert it to the correct representation. In the SYSCALL log, the hexadecimal version of the operation flags are found in the a2 segment, and extracted as an integer. Take 241, hexadecimal with 16 bits in the order:

0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	1

As seen, the flags in position 1, 7 and 10 are active (right to left). A bitwise *and* operation can be performed with the Unix syscall bits to decipher what exact syscalls are happening. The reason the results are asserted to be non-zero is that the result of the and-mask is still a denary number correspondent to the bit that was present. The utils module provides an operation enum to ensure consistent classification of these operations.

```
132 if flags & unix.O_PATH != 0 { op = utils.MetaData } else ...
```

The flags represented above correlate to:

- 0x200 - `O_TRUNC`
- 0x040 - `O_CREAT`
- 0x001 - `O_WRONLY`

As in: create the file if it doesn't exist, truncate the file if it does, and finally write.

Once everything is extracted from the SYSCALL log, the audit is appended to the slice, yet still incomplete. The PATH log contains an audit ID to be matched to the SYSCALL log, and details of the object that was involved. These details are parsed with regex and inserted into the correlating audit in state. The parsing of the SYSCALL and PATH logs is finished by `trackObject()` passing the required logic to the Subject struct's public method `AlterLabel()`.

5.5.3 Subject

The `AlterLabel()` method on Subject calculates if the label must be changed. As this is a proof of concept, the rules are pre-configured, such that primitive `if/else` statements are suitable to handle label transitions. Upon detecting the need for a label change, `restartSubject()` is called to complete the operation.

`selinux.Chcon()` is used to change the label of the entrypoint to the new label, decided by the previous method. The new label is spliced together with the fields of the old label, except `object_r`, which has to be given to the entrypoint, such that the spawned process can acquire `service_r`. A D-Bus connection is then created with `dbus.NewSystemConnectionContext()`, and used to restart the daemon, closed promptly after by Go's `defer` keyword.

That concludes the main execution cycle of the CWD. The rest of the functionality is maintained to provide more information at runtime.

5.5.4 Decorator and Logger

The `trackAVC()` branch of computation is used solely for logging purposes, and similar to much of the rest of the program, calls the `DecorateAndLog()` function of the decorator module when logging is required. While not the decorator pattern, this module serves to abstract logic that exists to make the production of the log file more readable. As such, `DecorateAndLog()` finishes by calling the `Log()` function of the logger module.

6 Verification & Validation

To provide a strong measure of success of the project, a rigorous testing suite was crucial to mark the success criteria as achieved.

6.1 Testing & Verification

The CWD underwent multiple types of tests, each serving a different primary purpose. Tests included:

- Integration Testing
- Unit Testing
- Benchmarking

6.1.1 Integration Testing

The integration testing suite was the primary source of confirmation during development. The suite consists of 6 simple daemons, written in C. They each attempted to read/write from different objects, some sporadically and some more consistently (see Appendix D.1). Evaluating the success of the CWD involved asserting that the correct labels were assigned under the correct circumstances. Each daemon had a specific label to halt on, ranging every different lattice route for the full Chinese Wall (see Appendix D.2). The logging feature was also heavily utilised, ensuring the CWD was receiving all the information needed, and scrutinising the information produced to trace errors.

6.1.2 Unit Testing

The Go standard library testing package was used to unit test the program. A testing suite was created for every module, which attempted to push the functionality to its limits. Line coverage was the metric used in unit testing, where the code is evaluated based on how many statements were executed in the testing suite. Testing was conducted until a high code coverage was achieved across the project, and all major modules were tested extensively.

6.1.3 Benchmarking

Benchmarking (performance testing) was conducted to investigate the speed of the CWD's enforcement. As will be discussed in limitations, the CWD lacks atomic label alterations, meaning that it cannot stop a subject from breaching `CoI` classes if they are instant. Therefore, the speed that the CWD *can* enforce on was tested, using a primitive testing suite written in Go. The suite involves repeatedly spawning a daemon, that reads from alpha, sleeps, and then writes the information it

read to beta. The time to sleep was decremented until it was able to write quick enough that it wasn't denied by the AVC after the CWD manipulated its label. The test ranged every step of $10\mu s$ between $5000\mu s - 0\mu s$, and marked the exact sleep intervals that the information managed to get leaked to beta. This process would be repeated 15 times for consistent results.

6.2 Validation

Validating the CWD involved returning to the requirements defined in the specification, and evaluating their success (see Table 4). Integration testing is abbreviated to "Int" where necessary.

Code	Requirement	Validation
FR1	`cwalld` must alter necessary configurations in the system before running.	Int & Unit
FR2	`cwalld` must monitor all operations made by subjects across a given directory.	Int & Unit
FR3	`cwalld` must make decisions based on subject label and object label aligned with the Chinese Wall Model.	Int & Unit
FR4	`cwalld` must alter the labels of subjects at upon call.	Int & Unit
FR5	`cwalld` must be accompanied by policy modules for objects.	Integration
FR6	`cwalld` must be accompanied by policy modules for subjects.	Integration
FR7	`cwalld` should be confined by SELinux with its own custom policy.	Integration
FR8	`cwalld` should run as a service in the system with a service declaration file.	Integration
FR9	`cwalld` should write all operations to a log.	Int & Unit
FR10	`cwalld` should include a program to follow and display the log in real time.	Int & Unit
FR11	`cwalld` could automatically create necessary policy modules for its runtime.	N/A
FR12	`cwalld` could automatically install necessary policy modules before running.	N/A
FR13	`cwalld` could dynamically enforce the Chinese Wall Model given a set of `CoI` classes as input.	N/A
FR14	`cwalld` won't enforce the Chinese Wall Model system-wide.	N/A

Table 4: Validating Functional Requirements

Non-functional requirements NFR1 - NFR4 could be evaluated by their own means. NFR5 (x86/ARM compatibility) was only tested on x86, as an ARM machine running SELinux was not obtained throughout the project. While measures were taken, it cannot be surmised that they would be enough to perfect the runtime similar to x86. While the requirement of NFR6 (atomic enforcement) was not realised, the benchmarking suite was established to assess the degree to which it was lacking.

7 Results & Evaluation

The evaluation of the CWD addresses test results and determines the success of the original project research question and objectives.

7.1 Evaluation Process

The most significant test result to evaluating the Chinese Wall is the integration testing, serving as the answer to the possibility of the Wall's implementation. Once the CWD minimum viable product was finished, the integration tests began to work. As more irregularities were detected and patched, the rates of success in the integration testing suite consistently rose. The integration testing suite now works as expected 100% of the time, even alongside standard system usage (see Appendix E).

Performance testing was the next most critical testing process, as it allowed the limitation of enforcement time to be analysed. It was important to dissect the speed of the enforcement, brought on by the various layers of processing for the CWD to perform. Across 15 test repetitions on the sleep intervals of $5000\mu s - 0\mu s$, a graph was constructed (see Appendix E.3) noting how frequent a breach was successful. The tests revealed that once past a threshold of $1880\mu s$ (of which the first occurrence of zero breaches was discovered), the failures became largely contained, with 97.43% of breach attempts being successfully contained. A single failure is noted as an outlier at $3850\mu s$, and the fastest enforcement speed was reported as $1110\mu s$.

Finally, unit testing was used to ensure robustness of the daemon throughout various different possible exceptional cases. A final code coverage of 71.3% was achieved, with the main modules of sleuth and subject at 64.8% and 89.4% respectively.

7.2 Results of Evaluation

The results of both performance and unit testing will be discussed, before moving to integration testing, which directly correlates to the answer to the research question and main objective.

The results of the benchmark indicate that the CWD can enforce the Chinese Wall very quickly. While not atomic, the example of a userspace daemon acting alongside multiple layers of computation was pushed to its absolute limit, with the enforcement being consistently effective after 1.88ms. This proves the CWD was made with speed in mind, and uses best practice to ensure it can run in the

background at minimal cost overhead.

The high code coverage of unit testing was not only useful to test large components but brought minor bugs to light to be patched. The CWD is proven to be robust by these means.

7.3 Returning to the Objectives

7.3.1 Research Question

"Can SELinux's Mandatory Access Control system facilitate the Chinese Wall Policy?" While the other tests did not target the research question, the success of the integration tests provide a direct answer to the achieved implementation of the Chinese Wall model. The Chinese Wall was implemented, true to the properties of the Brewer-Nash Model and utilising the MAC system of SELinux. The project can be considered a success, as it proved that it is possible to implement the Brewer-Nash Model in SELinux.

7.3.2 Objective 1

"Develop a set of Policy Modules, capable of enforcing a static Chinese Wall on subjects confined by SELinux." The static implementation of the Chinese Wall was completed on schedule and appropriately guided the success of the following stage. Validation of this stage was concluded in the project progress report presentation.

7.3.3 Objective 2

"Implement a method of dynamically altering labels, allowing SELinux to enforce the Chinese Wall policy." The dynamic Chinese Wall was fully implemented and tested, according to the functional requirements set out, and is proven to work as intended across multiple circumstances with the validation techniques previously listed.

7.3.4 Objective 3

"Conclude why development in this area may be lacklustre, by analysing the difficulties and limitations that arose with development." As objective 3 is mainly a reflective objective, it will be addressed in the following section, reflecting on the challenges and limitations of the Chinese Wall.

8 Discussion & Reflection

8.1 Interpreting the Results

The lack of Chinese Wall inclusion in SELinux is still unclear. As shown, the development of a Chinese Wall in SELinux is certainly possible, with the potential of a thorough security measure being possible with more work (as noted in limitations). The most likely reason for the lack of Chinese Wall attention is a lack of practical use case. Adjacent implementations included specific problems to be solved, of which SELinux lacks that requirement. Most commonly in practice, SELinux would run as a VM on a cloud server, as an end user product. The nature of `CoI` may pose a threat to SELinux users, but the threat model is lacking in scale or necessity.

Regardless, the results of the validation means all exemplify the success of the project. The answer to the research question is: **Yes**, a Chinese Wall in SELinux can be created, and the proof is a culmination of all the work in this paper. The Chinese Wall was created, and while not a perfect security daemon, it absolutely can be done.

8.2 Reflection

A factor holding the project back was a lack of in depth research of SELinux policy modules, and patience while writing them. Had more resources been scoured earlier, development would have started later, but on a cleaner and more confident footing.

Having more faith that the project would work out would have also promoted healthy progress, as certain setbacks in the project led to loss of confidence for a successful implementation.

8.3 Challenges

While popular, SELinux severely lacks detailed documentation for developers creating policy modules. This proved it to be a challenging environment to navigate, along with compilation errors that were punishing and not explanatory, leading to frustration and clouded decision-making.

Entering the SELinux environment also proved challenging, as there were many available avenues for experimenting. Navigating these were often completely unyielding and made early progress feel vain, as the entire venture would be pivoted from.

Another challenge was surrounding the nature of a security daemon. As more features were integrated

into SELinux, it became increasingly difficult to understand what part of the program was failing. Errors would propagate seemingly randomly, of which the resolution was completely elsewhere from recent changes.

8.4 Limitations

The most crucial limitation of the project involves layers of computation required to enforce label alterations. Due to the nature of the daemon being entirely user-space, the daemon lacks the ability to atomically alter labels. Unfortunately, it would be rather useless as a security daemon, as it lacks the ability to stop a subject from writing after reading, spreading information. A kernel hook could circumvent this, integrating with the daemon such that the labels can be enforced instantaneously. This would however require kernel level programming, and was deemed to be outside the scope of an undergraduate project.

Another limitation takes form in SELinux requiring a context restart to apply labels. While most system daemons are able to terminate and restart easily, this could have unforeseen issues with other subjects. Unfortunately, this limitation is unavoidable and is a constraint built directly into SELinux.

8.5 Future Work

Future work would certainly involve kernel level restrictions. eBPF for one, is a kernel technology that was considered in case the kernel approach was to be taken. Kernel integration would surpass the proof of concept of the daemon, and make it a legitimate security daemon capable of enforcing the Brewer-Nash Model.

Another avenue of work could involve automatic `CoI` detection and policy installation. It would not pose as much of a challenge and/or benefit as kernel integration, but would definitely improve quality of the product. The Aggressive Chinese Wall Model could also be addressed, featuring a `CoI` matrix of the CWD's own, instead of the primitive label flow logic build into the current rendition.

9 Conclusion

The aim of this project was to prove the feasibility of the Chinese Wall Model in Security-Enhanced Linux, by proof of construction. It detailed the complicated design process that preceded the creation of a Chinese Wall true to the Brewer-Nash Model in SELinux. Beginning the project in a personally uncharted environment, testing and experimenting with the environment led to the creation of the static wall, capturing the Brewer-Nash Model's restrictions in a stationary setting.

Related successes with adjacent Chinese Walls fuelled inspiration for a true dynamic implementation. Methods would be found to enforce dynamic label changes, with the prototyping and design of the Chinese Wall Daemon to follow. There would be many issues tackled while developing the CWD, all of which would eventually be resolved with it undergoing extensive testing to ensure it achieved its objectives.

The completion of the CWD marks an achievement in navigating and leveraging the challenging SELinux environment to conform to a dynamic system. Considerable effort went into producing the CWD, and it will likely remain a niche contribution to the SELinux world, but the challenge to implement it was a genuinely rewarding pursuit.

References

[1] D. Brewer and M. Nash, "The chinese wall security policy," in *Proceedings. 1989 IEEE Symposium on Security and Privacy*, pp. 206–214, IEEE Comput. Soc. Press.

[2] "Red hat enterprise linux | red hat partner connect."

[3] D. E. Bell and L. J. Lapadula, "Secure computer systems: Mathematical foundations," Number: MTR2547VOL1.

[4] T. Y. Lin, "Chinese wall security policy-an aggressive model," pp. 282,283,284,285,286,287,288,289–282,283,284,285,286,287,288,289, IEEE Computer Society.

[5] S. N. Foley, "Building chinese walls in standard unixTM," vol. 16, no. 6, pp. 551–563.

[6] Y. Fan, Z. Han, J. Liu, and Y. Zhao, "A mandatory access control model with enhanced flexibility," in *2009 International Conference on Multimedia Information Networking and Security*, vol. 1, pp. 120–124. ISSN: 2162-8998.

[7] R. Sailer, T. Jaeger, E. Valdez, R. Caceres, R. Perez, S. Berger, J. Griffin, and L. van Doorn, "Building a MAC-based security architecture for the xen open-source hypervisor," in *21st Annual Computer Security Applications Conference (ACSAC'05)*, pp. 10 pp.–285. ISSN: 1063-9527.

[8] T.-H. Tsai, Y.-C. Chen, H.-C. Huang, P.-M. Huang, and K.-S. Chou, "A practical chinese wall security model in cloud computing," in *2011 13th Asia-Pacific Network Operations and Management Symposium*, pp. 1–4.

[9] S. Yu, X. Gui, J. Lin, F. Tian, J. Zhao, and M. Dai, "A security-awareness virtual machine management scheme based on chinese wall policy in cloud computing," vol. 2014, no. 1, p. 805923. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1155/2014/805923.

[10] Y.-C. Hsiao and G.-H. Hwang, "Implementing the chinese wall security model in workflow management systems," in *International Symposium on Parallel and Distributed Processing with Applications*, pp. 574–581. ISSN: 2158-9208.

[11] L. Herzel and D. E. Colling, "The chinese wall and conflict of interest in banks," vol. 34, no. 1, pp. 73–115. Publisher: American Bar Association.

[12] S. Smalley, "Configuring the SELinux policy,"

[13] K. A. Khan, M. Amin, A. K. Afridi, and W. Shehzad, "SELinux in and out," in *2011 IEEE 3rd International Conference on Communication Software and Networks*, pp. 339–343.

[14] S. Smalley and T. Fraser, "A security policy confguration for the security-enhanced linux,"

[15] P. A. Loscocco and S. D. Smalley, "Meeting critical security objectives with security-enhanced linux,"

[16] L. T. Koehler, "Linux: An operating system," vol. 5, no. 1, pp. 136–141. Num Pages: 6.

[17] B. Sarna-Starosta and S. D. Stoller, "Policy analysis for security-enhanced linux,"

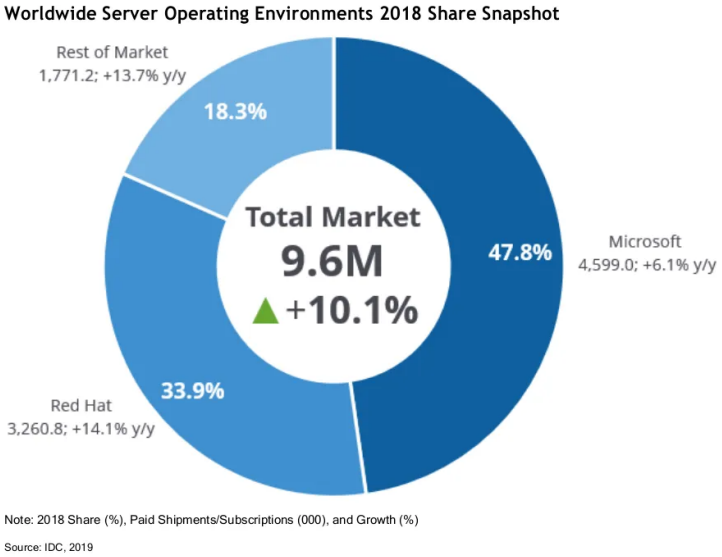
[18] H. James, "85% of all smartphones are powered by linux."

[19] "Usage statistics and market share of linux for websites, march 2026."

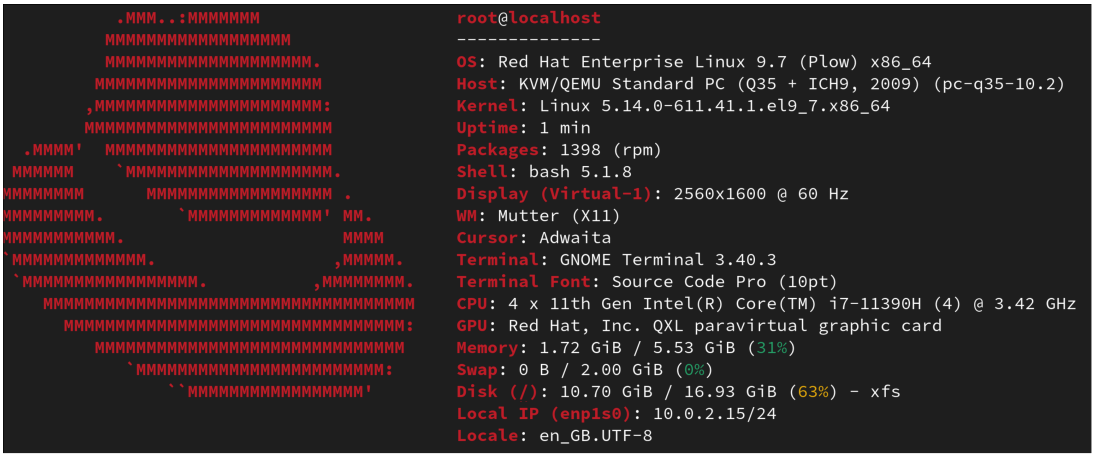
[20] "Red hat: Leading the enterprise linux server market."

A Appendix - Red Hat

A.1 Red Hat Details



A.2 Virtualised Red Hat system specifications



↑ Return to Literature Review ↑ Return to Implementation

B Appendix - Design

B.1 Auditd logs

B.2 Auditd SYSCALL log

```
type=SYSCALL msg=audit(1774217186.548:210): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c
a1=402018 a2=241 a3=1b6 items=2 ppid=1 pid=3299 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egi
d=0 sgid=0 fsgid=0 tty=(none) ses=4294967295 comm="alpha_civild" exe="/usr/local/bin/alpha_civild" su
bj=system_u:system_r:unconfined_service_t:s0 key="cwalld"
```

B.2.1 Auditd PATH log

```
type=PATH msg=audit(1774217339.211:13879): item=0 name="/home/testgrounds/objects/alpha_logs" inode=5
8142854 dev=fd:00 mode=0100666 ouid=1000 ogid=1000 rdev=00:00 obj=unconfined_u:object_r:alpha_t:s0 na
metype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
```

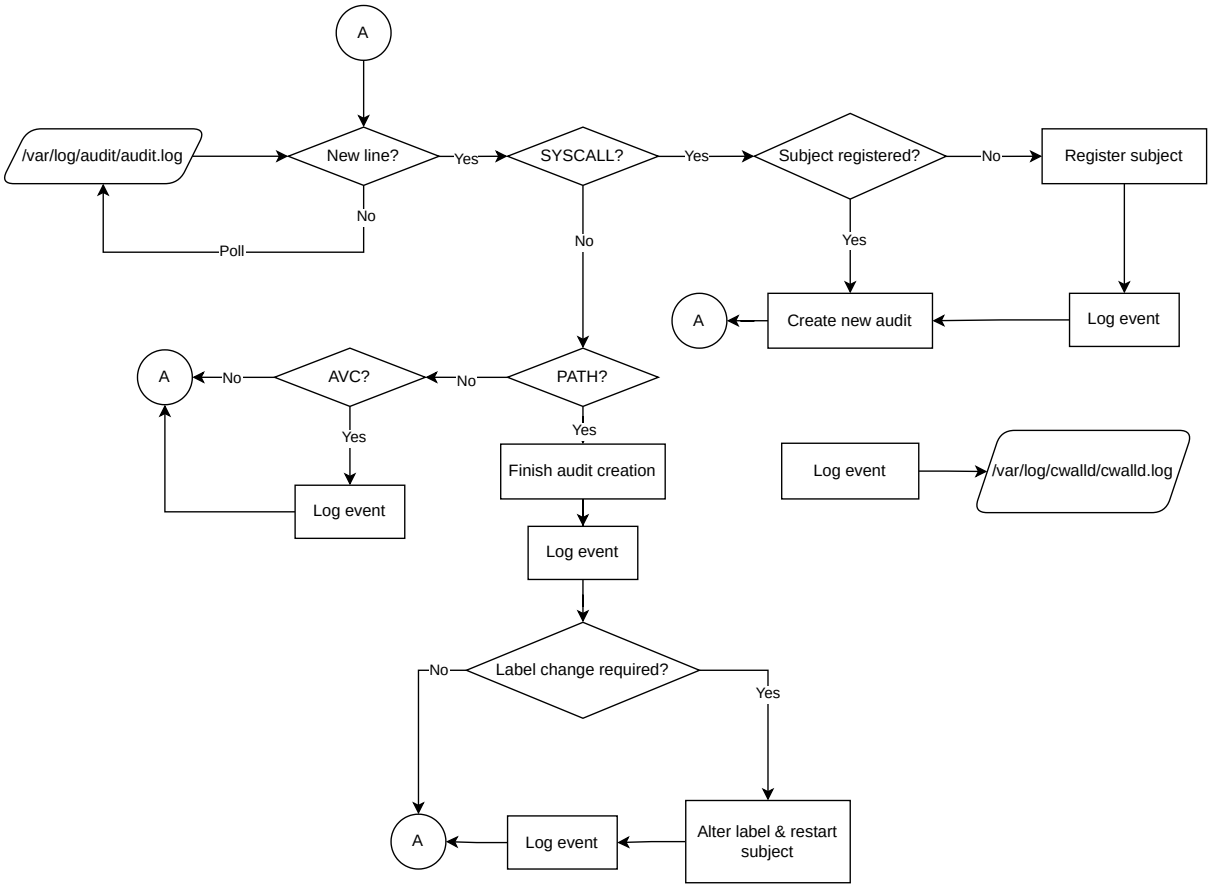
B.2.2 Auditd AVC log

```
type=AVC msg=audit(1774217334.204:13868): avc:  denied  { read } for  pid=3485 comm="beta_evild" name
="alpha_logs" dev="dm-0" ino=58142854 scontext=system_u:system_r:beta_rw_t:s0 tcontext=unconfined_u:o
bject_r:alpha_t:s0 tclass=file permissive=0
```

↑ Return to Design

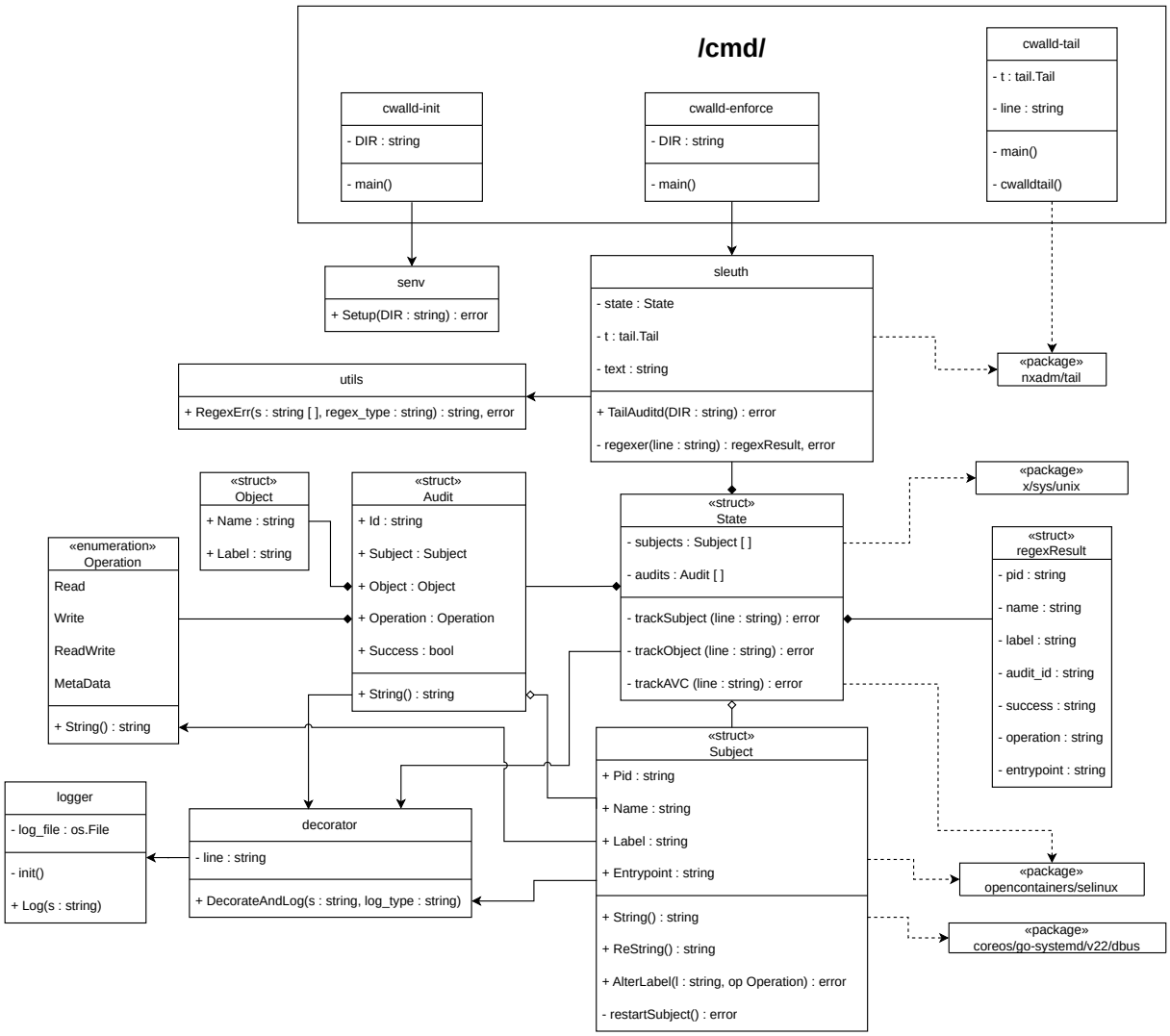
B.3 CWD Design Diagrams

B.3.1 CWD Flowchart



↑ Return to Design

B.3.2 CWD UML



↑ Return to Design

C Appendix - Implementation

C.1 Full Subject Policy Module (beta_rw)

```
policy_module(beta_rw, 1.0.0)

########################################
#
# Declarations
#
type beta_rw_t;
type beta_rw_exec_t;

require {
    type beta_t;
    type gamma_t;
    type sanitised_t;
    type user_home_t;
    type user_home_dir_t;
}

init_daemon_domain(beta_rw_t, beta_rw_exec_t)

########################################
#
# beta_rw local policy
#
allow beta_rw_t self:fifo_file rw_fifo_file_perms;
allow beta_rw_t self:unix_stream_socket create_stream_socket_perms;

# cwalld rules
allow beta_rw_t beta_t:file { read write getattr open };
allow beta_rw_t gamma_t:file { read getattr open };
allow beta_rw_t sanitised_t:file { read getattr open };

# navigational policies
```

```
allow beta_rw_t user_home_t:dir { search read open write add_name };
allow beta_rw_t user_home_dir_t:dir { search read open write
    add_name };
```

↑ Return to Implementation

C.2 Chinese Wall Daemon Policy Module

```
policy_module(cwalld, 1.0.0)

type cwalld_t;
type cwalld_exec_t;

require {
    type proc_t;
    type alpha_rw_exec_t;
    type beta_rw_exec_t;
    type gamma_rw_exec_t;
    type sysfs_t;
    type bin_t;
    type unconfined_t;
    type system_dbusd_t;
    type system_dbusd_var_run_t;
    type unconfined_service_t;
    type init_t;
    type systemd_unit_file_t;
    type unlabeled_t;
    type auditd_log_t;
    type cgroup_t;
    type user_home_t;
    type var_log_t;
    type sanitised_t;
    class process getattr;
    class capability { dac_read_search sys_admin sys_ptrace };
    class file { append execute execute_no_trans getattr map open
        read relabelfrom relabelto watch };
```

```
    class dir { getattr search };
    class lnk_file read;
    class filesystem mount;
    class sock_file write;
    class unix_stream_socket connectto;
    class dbus send_msg;
    class service start;
}

init_daemon_domain(cwalld_t, cwalld_exec_t)

#============= cwalld_t ==============
allow cwalld_t auditd_log_t:dir search;
allow cwalld_t auditd_log_t:file { getattr open read watch };
allow cwalld_t bin_t:file relabelfrom;
allow cwalld_t cgroup_t:dir search;
allow cwalld_t cgroup_t:file { open read };
allow cwalld_t init_t:dbus send_msg;
allow cwalld_t proc_t:filesystem mount;
allow cwalld_t self:capability { dac_read_search dac_override
    sys_admin sys_ptrace };
allow cwalld_t self:process getattr;
allow cwalld_t sysfs_t:file { open read };
allow cwalld_t system_dbusd_t:dbus send_msg;
allow cwalld_t system_dbusd_t:unix_stream_socket connectto;
allow cwalld_t system_dbusd_var_run_t:dir search;
allow cwalld_t system_dbusd_var_run_t:sock_file write;
allow cwalld_t systemd_unit_file_t:service start;
allow cwalld_t var_log_t:dir { create write add_name };
allow cwalld_t var_log_t:file { create write open getattr setattr };
allow cwalld_t unconfined_service_t:dir search;
allow cwalld_t unconfined_service_t:file { open read relabelfrom
    relabelto };
allow cwalld_t unconfined_service_t:lnk_file read;
allow cwalld_t unconfined_service_t:process getattr;
```

```
allow cwalld_t unconfined_t:dir search;
allow cwalld_t unconfined_t:file { open read };
allow cwalld_t unconfined_t:lnk_file read;
allow cwalld_t unconfined_t:process getattr;
allow cwalld_t user_home_t:dir { getattr search };

allow cwalld_t file_type:file { relabelto relabelfrom setattr getattr };
allow cwalld_t file_type:dir { relabelto relabelfrom setattr getattr };
allow cwalld_t file_type:lnk_file { relabelto relabelfrom setattr getattr };
allow unconfined_t cwalld_exec_t:file { read execute open getattr };
allow cwalld_t domain:file { read open getattr };
allow cwalld_t domain:dir { search getattr };
allow cwalld_t domain:lnk_file read;
allow cwalld_t domain:process getattr;

#============= init_t ==============
allow init_t auditd_log_t:file { open read watch };
allow init_t cwalld_t:dbus send_msg;
```

↑ Return to Implementation

D Appendix - Testing Suite

D.1 Integration Testing Daemon

```
#include <unistd.h>
#include <stdio.h>

FILE *f;

// confused gamma daemon
// it will align itself with gamma, then read from beta, then try to read from alpha as well

int main(void){
  while(1){
    // allowed - should have label unconfined_service_t / gamma_rw_t
    f = fopen("/home/testgrounds/objects/gamma_reports", "r");
    sleep(3);
    if (f != NULL) {
      fclose(f);
    }
    sleep(1);
    // allowed - should have label beta_gamma_r_t
    f = fopen("/home/testgrounds/objects/beta_plans", "r");
    sleep(3);
    if (f != NULL) {
      fclose(f);
    }
    sleep(1);
    // denied - label beta_gamma_r_t should not allow
    f = fopen("/home/testgrounds/objects/alpha_logs", "r");
    sleep(3);
    if (f != NULL) {
      fclose(f);
    }
    sleep(1);
```

```
    // allowed
    f = fopen("/home/testgrounds/objects/sanitised", "r");
    sleep(3);
    if (f != NULL) {
      fclose(f);
    }
    sleep(1);
  }
}
```

D.2 Integration Testing Plan

There are seven daemons, with their transitions noted:

- alpha_civild → alpha_rw_t
- alpha_curiousd → alpha_rw_t → alpha_gamma_t
- beta_civildd → beta_rw_t
- beta_evild → beta_rw_t → beta_gamma_t
- gamma_civild → gamma_rw_t
- gamma_confusedd → gamma_rw_t → beta_gamma_t
- unconfined_daemond → stay as unconfined_service_t

The test is a marked a success if all daemons acquire their correct labels.

↑ Return to Testing

E Appendix - Test Results

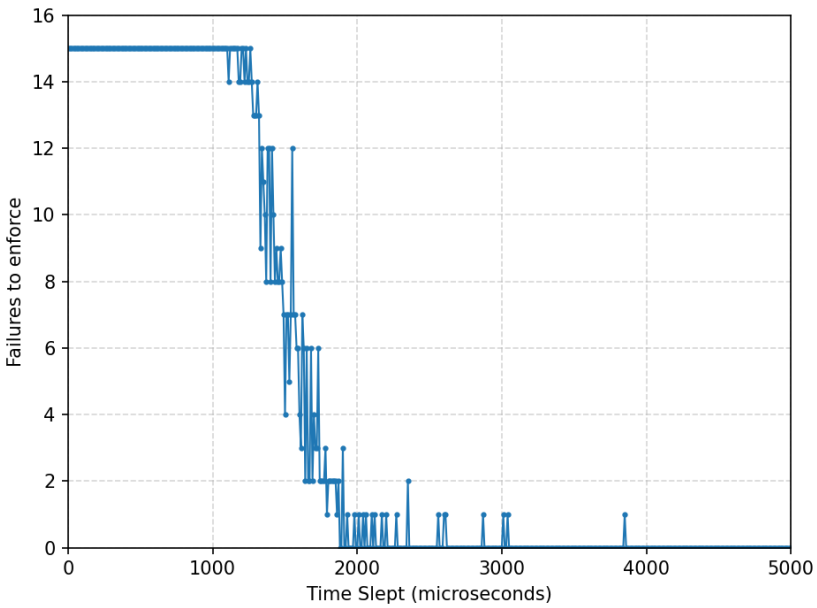
E.1 Before Integration Testing

```
/usr/local/bin ❯ ls -Zl | grep system_u
root root system_u:object_r:bin_t:s0     17576 Mar 10 18:18 alpha_civild
root root system_u:object_r:bin_t:s0     17584 Mar 14 17:51 alpha_curiousd
root root system_u:object_r:bin_t:s0     17576 Mar 10 18:35 beta_civild
root root system_u:object_r:bin_t:s0     17576 Mar 14 17:47 beta_evild
root root system_u:object_r:bin_t:s0     17576 Mar 10 18:37 gamma_civild
root root system_u:object_r:bin_t:s0     17584 Mar 10 18:54 gamma_confusedd
root root system_u:object_r:bin_t:s0     17584 Mar 28 15:07 unconfined_daemond
```

E.2 After Integration Testing

```
/usr/local/bin ❯ ls -Zl | grep system_u
root root system_u:object_r:alpha_rw_exec_t:s0      17576 Mar 10 18:18 alpha_civild
root root system_u:object_r:alpha_gamma_r_exec_t:s0 17584 Mar 14 17:51 alpha_curiousd
root root system_u:object_r:beta_rw_exec_t:s0       17576 Mar 10 18:35 beta_civild
root root system_u:object_r:beta_gamma_r_exec_t:s0  17576 Mar 28 15:22 beta_evild
root root system_u:object_r:gamma_rw_exec_t:s0      17576 Mar 10 18:37 gamma_civild
root root system_u:object_r:beta_gamma_r_exec_t:s0  17584 Mar 10 18:54 gamma_confusedd
root root system_u:object_r:bin_t:s0                17584 Mar 28 15:07 unconfined_daemond
```

E.3 Enforcement Failure Graph



↑ Return to Evaluation

E.4 Unit Testing Coverage

```
cal@localhost ~/cs408-cwalld/cwalld main* ❯ sudo go test ./.../. --cover
        cwalld/cmd/cwalld/cwalld-enforce                coverage: 0.0% of statements
        cwalld/cmd/cwalld/cwalld-init           coverage: 0.0% of statements
ok      cwalld/cmd/cwalld/cwalld-tail   (cached)        coverage: 57.1% of statements
ok      cwalld/internal/audit   (cached)        coverage: 100.0% of statements
ok      cwalld/internal/decorator       (cached)        coverage: 100.0% of statements
ok      cwalld/internal/logger  (cached)        coverage: 66.7% of statements
ok      cwalld/internal/senv    (cached)        coverage: 84.6% of statements
ok      cwalld/internal/sleuth  (cached)        coverage: 64.8% of statements
ok      cwalld/internal/subject (cached)        coverage: 89.4% of statements
ok      cwalld/internal/utils   (cached)        coverage: 100.0% of statements
```

```
cal@localhost ~/cs408-cwalld/cwalld main* ❯ go tool cover -func out.coverage
cwalld/cmd/cwalld/cwalld-enforce/main.go:8:     main            0.0%
cwalld/cmd/cwalld/cwalld-init/main.go:9:        main            0.0%
cwalld/cmd/cwalld/cwalld-tail/main.go:11:       main            0.0%
cwalld/cmd/cwalld/cwalld-tail/main.go:15:       cwalldtail      66.7%
cwalld/internal/audit/audit.go:17:              String          100.0%
cwalld/internal/decorator/decorator.go:21:      DecorateAndLog  100.0%
cwalld/internal/logger/logger.go:12:            init            60.0%
cwalld/internal/logger/logger.go:32:            Log             100.0%
cwalld/internal/senv/senv.go:10:                Setup           84.6%
cwalld/internal/sleuth/sleuth.go:34:            TailAuditd      0.0%
cwalld/internal/sleuth/sleuth.go:78:            trackSubject    67.5%
cwalld/internal/sleuth/sleuth.go:149:           trackObject     87.5%
cwalld/internal/sleuth/sleuth.go:192:           trackAVC        69.0%
cwalld/internal/sleuth/sleuth.go:242:           regexer         79.6%
cwalld/internal/subject/subject.go:22:          String          100.0%
cwalld/internal/subject/subject.go:26:          ReString        100.0%
cwalld/internal/subject/subject.go:30:          AlterLabel      89.3%
cwalld/internal/subject/subject.go:84:          restartSubject  88.2%
cwalld/internal/utils/utils.go:22:              String          100.0%
cwalld/internal/utils/utils.go:36:              RegexErr        100.0%
total:                                          (statements)    71.3%
```

↑ Return to Evaluation